PDE and Physics-Informed learning

1 Physics-Informed Neural Networks

1.1 Problem statement for 2D direct problems

Consider a partial differential equation (PDE) written in the following residual form as,

$$\mathcal{F}(u, x, y, u_x, u_y, ...) = 0, \quad (x, y) \in \Omega, \tag{1}$$

where $u(x, y)$ denotes the desired solution and u_x, u_y, ... are the required associated partial derivatives of different orders with respect to x and y. Specific conditions must be also imposed at the domain boundary $\partial\Omega$ depending on the problem.

Supervised learning approach using training data. The classical supervised learning approach assumes that we have at our disposal a dataset of N_{data} known input-output pairs (x, u):

$$\mathcal{D} = \{(x_i^{data}, u_i^{data})\}_{i=1}^{N_{data}},$$

for $i \in 1, N_{data}$. u_θ is considered to be a good approximation of u if predictions $u_\theta(x_i)$ are close to target outputs u_i^{data} for every data samples i. We want to minimize the prediction error on the dataset, hence it's natural to search for a value $\theta^\star$ solution of the following optimization problem:

$$\theta^\star = \underset{\theta}{\operatorname{argmin}}\, L_{data}(\theta) \tag{2}$$

with

$$L_{data}(\theta) = \frac{1}{N_{data}} \sum_{i=1}^{N_{data}} \left|(u_\theta(x_i) - u_i^{data}\right|^2. \tag{3}$$

1.2 PINNs for solving PDEs

In PINNs approach, a specific loss function L_{PDE} can be defined as,

$$L_{PDE}(\theta) = \frac{1}{N_c} \sum_{i=1}^{N_c} \left|\mathcal{F}(u_\theta(x_i))\right|^2, \tag{4}$$

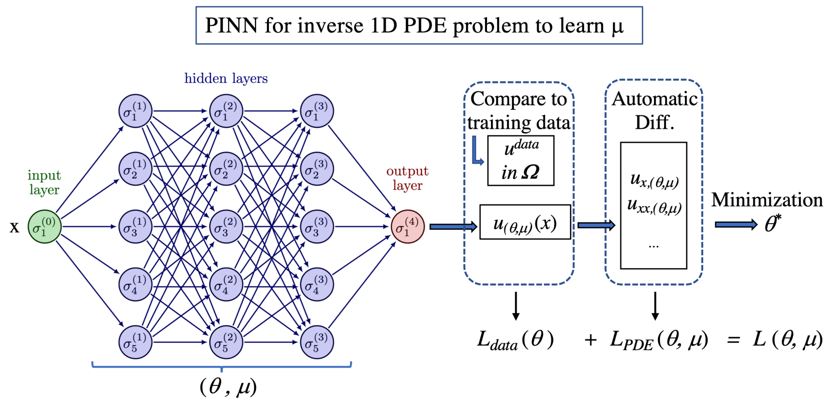

Figure 1: Schematic representation of an inverse problem. A 1D differential equation is considered (ODE in fact) parametrized with an unknown coefficient μ to be discovered. The training data set must include data from the whole domain Ω (not only at boundaries).

where the evaluation of the residual equation is performed on a set of N_c points denoted as x_i. These points are commonly referred to as collocation points. A composite total loss function is typically formulated as follows

$$L(\theta) = \omega_{data} L_{data}(\theta) + \omega_{PDE} L_{PDE}(\theta), \tag{5}$$

where ω_{data} and ω_{PDE} are weights to be assigned to ameliorate potential imbalances between the two partial losses. These weights can be user-specified or automatically tuned. A schematic representation of the architecture of a neural network designed to solve a PDE using PINNs is visible in Fig. 1. For simplicity, the weights are taken to be unity in this example.This scheme corresponds to a direct problem involving involving Neumann BCs or mixed Neumann-Dirichlet BCs.

References

[1] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. *Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations.* 2017. arXiv: `1711.10561 [cs.AI]`. URL: `https://arxiv.org/abs/1711.10561`.